

Programming in Python

A Comprehensive Textbook for Data Science Students

IIT Madras BS Degree Programme

IITM BS Degree Curriculum Team

August 4, 2026

Contents

Introduction	ii
1 Introduction to Python, Variables, Data Types & Operators	1
1.1 Variables and Literals	1
1.2 Core Primitive Data Types	2
1.3 String Slicing and Basic String Operations	2
1.4 Operators and Expressions	3
1.5 Practice Exercises	4
2 Control Flow & Conditionals	5
2.1 Conditional Statements (if, elif, else)	5
2.2 Nested Conditionals and Short-Circuit Evaluation	6
2.3 Importing Standard Libraries	6
2.4 Practice Exercises	7
3 Loops & Iteration	8
3.1 The while Loop	8
3.2 The for Loop and range() Function	8
3.3 Nested Loops	9
3.4 Loop Control Statements: break, continue, and pass	10
3.5 Practice Exercises	11
4 Lists & Basic Matrix Algorithms	12
4.1 List Creation, Indexing, and Mutability	12
4.2 Linear Search and Basic Vector Operations	13
4.3 2D Lists and Matrix Algorithms	13
4.4 Selection Sort ("Obvious Sort")	14
4.5 Practice Exercises	15
5 Tuples, Dictionaries & Sets	16
5.1 Tuples: Immutable Sequences	16
5.2 Dictionaries: Key-Value Mapping	16
5.3 Sets: Unordered Unique Collections	17
5.4 Practice Exercises	18
6 Functions, Arguments & Scope	19
6.1 Defining and Calling Functions	19
6.2 Types of Function Arguments	19
6.3 Scope of Variables (LEGB Rule)	20
6.4 Practice Exercises	21

7	Functional Programming, Comprehensions & Generators	22
7.1	List, Dictionary, and Set Comprehensions	22
7.2	Lambda Functions, <code>map()</code> , <code>filter()</code> , <code>zip()</code> , and <code>enumerate()</code>	23
7.3	Generators and the <code>yield</code> Keyword	23
7.4	Practice Exercises	25
8	File Handling & Exception Management	26
8.1	Reading and Writing Files	26
8.2	Processing Large Datasets	26
8.3	Exception Handling (<code>try</code> , <code>except</code> , <code>else</code> , <code>finally</code>)	27
8.4	Practice Exercises	28
9	Recursion & Searching Algorithms	29
9.1	The Core Mechanics of Recursion	29
9.2	Binary Search: Iterative vs Recursive	29
9.3	Practice Exercises	31
10	Object-Oriented Programming (OOP)	32
10.1	Classes, Objects, <code>__init__</code> , <code>__str__</code> , and <code>self</code>	32
10.2	Inheritance and Method Overriding	33
10.3	Practice Exercises	34
11	Scientific Computing: NumPy & Pandas	35
11.1	NumPy Arrays vs Python Lists	35
11.2	Pandas Series and DataFrames	35
11.3	Practice Exercises	37
12	Data Visualization & Course Summary	38
12.1	Introduction to Matplotlib	38
12.2	Common Chart Types for Data Science	39
12.3	Course Summary and Next Steps	39
12.4	Practice Exercises	40
	Solutions to Practice Exercises	42

Introduction

Welcome to the **Programming in Python** Comprehensive Textbook for the IIT Madras BS Degree Programme. This textbook is designed to build software engineering and computational thinking skills from ground-level Python fundamentals through to scientific computing with NumPy, Pandas, and Matplotlib.

Structure of this Guide

The course is organized into **12 Chapters**:

- **Chapters 1–4 (Quiz 1 Content):** Variables, Data Types, Control Flow, Loops, Lists, and Basic Algorithmic Logic.
- **Chapters 5–8 (Quiz 2 Content):** Data Structures (Tuples, Dictionaries, Sets), Functions, Functional Programming, Comprehensions, File Handling, and Exception Management.
- **Chapters 9–12 (End-Term Content):** Recursion, Searching Algorithms, Object-Oriented Programming (OOP), NumPy, Pandas, and Data Visualization.

Visual Layout & Box Legend

- **Teal Cards** — Core concepts and terminology.
- **Slate Blue Cards** — Python syntax rules and language properties.
- **Purple Cards** — Code examples and execution walk-throughs.
- **Violet Cards** — Core intuition and computational memory models.
- **Amber Cards** — Key programming patterns and algorithmic logic.
- **Red Cards** — Video resources and lecture links.

Chapter 1

Introduction to Python, Variables, Data Types & Operators

Welcome to Programming in Python! Python is the undisputed language of modern Data Science, Machine Learning, and Artificial Intelligence. Its clean, human-readable syntax makes it easy to write code, while its powerful ecosystem of libraries (like NumPy, Pandas, and PyTorch) allows data scientists to build cutting-edge intelligent systems.

1.1 Variables and Literals

In Python, a **variable** is a named location in memory used to store data. Unlike languages like C++ or Java, Python uses **dynamic typing**, meaning you do not need to explicitly declare a variable's data type when creating it.

Concept: Variables and Assignment

A variable is created the moment you first assign a value to it using the assignment operator (=).

```
x = 10          # An integer variable
name = "IITM"   # A string variable
pi = 3.14159     # A floating-point variable
is_valid = True  # A boolean variable
```

Data Science Relevance: In data science, variables hold feature columns, target labels, model hyperparameters (like learning rate $\alpha = 0.01$), and dataset paths.

Video Resource: IITM BS Lecture: Introduction to Variables



Official lecture introducing Python variables, assignment statements, and memory concepts.

Video Resource: IITM BS Lecture: Variables and Literals



Official lecture explaining literals, variable naming rules, and assignment mechanics.

1.2 Core Primitive Data Types

Python has four primary built-in primitive data types:

Concept: Primitive Data Types

- **Integer (int):** Whole numbers without decimals (e.g., -5, 0, 42).
- **Float (float):** Real numbers with decimal points or scientific notation (e.g., 3.14, -0.001, 2.5e-4).
- **String (str):** Sequences of text characters enclosed in single, double, or triple quotes (e.g., "data", 'Python').
- **Boolean (bool):** Truth values: True or False.

You can inspect the type of any variable using the built-in `type()` function.

Code Example: Checking Types and Type Casting

```
age = 25
print(type(age))  # Output: <class 'int'>

# Type Conversion (Type Casting)
height_str = "1.75"
height_float = float(height_str)  # Converts string "1.75" to float 1.75
print(type(height_float))         # Output: <class 'float'>
```

Video Resource: IITM BS Lecture: Data Types (Parts 1 & 2)



Official lectures covering numeric data types, type conversion, and character strings.

1.3 String Slicing and Basic String Operations

Strings are immutable sequences of characters. You can access individual characters using 0-based indexing or slice sub-strings using the bracket syntax `[start:stop:step]`.

Key Pattern / Logic: String Slicing Syntax

Given `s = "DataScience"`:

- `s[0] → 'D'` (First character)
- `s[-1] → 'e'` (Last character via negative indexing)
- `s[0:4] → 'Data'` (Slice from index 0 up to, but not including, index 4)
- `s[::-1] → 'ecneicSataD'` (Reverse the string)

Video Resource: IITM BS Lecture: Strings and String Slicing 

Official lecture demonstrating string indexing, negative indices, and slicing notation.

1.4 Operators and Expressions

Python supports arithmetic, comparison (relational), logical, and assignment operators.

Syntax / Rule: Arithmetic and Logical Operators

- **Arithmetic:** + (add), - (subtract), * (multiply), / (float division), // (floor division), % (modulus / remainder), `\{\}\texttt{b}\{\}` (exponentiation).
- **Comparison:** == (equal), != (not equal), >, <, >=, <=.
- **Logical:** and, or, not.

Code Example: Floor Division vs Division

```
a = 17
b = 5

print(a / b)    # 3.4      (Float Division)
print(a // b)   # 3       (Floor Division: drops decimal part)
print(a % b)    # 2       (Modulus: remainder of 17 / 5)
print(a ** b)   # 1419857 (17 raised to the power of 5)
```

Video Resource: IITM BS Lecture: Operators and Expressions 

Official lecture detailing operator precedence, modulus, floor division, and logical expressions.

1.5 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **Variable Types:** Identify the data type (`int`, `float`, `str`, or `bool`) of the following Python expressions:

- (a) `x = 10 / 2`
- (b) `y = 10 // 2`
- (c) `z = "100" + "200"`
- (d) `w = (5 > 3) and (2 == 4)`

2. **Arithmetic Logic:** Calculate the exact numerical output of the following Python expressions without running the code:

- (a) `2 + 3 * 4 ** 2`
- (b) `25 % 7`
- (c) `19 // 4`

3. **String Slicing:** Given the string `text = "MachineLearning"`:

- (a) Write a slice to extract "Machine".
- (b) Write a slice to extract "Learning".
- (c) What is the output of `text[-8:]`?
- (d) Write a slice to reverse the entire string.

4. **Type Casting & Input:** A user enters their age as a string "22". Write a 2-line Python script that converts this input into an integer, adds 5 years, and prints: "In 5 years, you will be 27".

5. **Data Science Application:** You are calculating the Body Mass Index (BMI) of a patient. The formula is:

$$\text{BMI} = \frac{\text{weight in kg}}{(\text{height in meters})^2}$$

Given `weight = 75` and `height = 1.75`, write a Python snippet to compute the BMI and round it to 2 decimal places using `round(bmi, 2)`.

Chapter 2

Control Flow & Conditionals

Programs need to make decisions based on changing data. In Python, **Control Flow** allows you to execute specific blocks of code depending on whether a given condition evaluates to `True` or `False`.

2.1 Conditional Statements (`if`, `elif`, `else`)

Python uses indentation (4 spaces) to define code blocks instead of curly braces `{ }`.

Syntax / Rule: Syntax of Conditional Blocks

```
if condition_1:
    # Executed if condition_1 is True
elif condition_2:
    # Executed if condition_1 is False AND condition_2 is True
else:
    # Executed if all above conditions are False
```

Code Example: Categorizing Credit Scores

```
credit_score = 720

if credit_score >= 750:
    status = "Excellent"
elif credit_score >= 650:
    status = "Good"
elif credit_score >= 550:
    status = "Fair"
else:
    status = "Poor"

print(f"Credit Status: {status}") # Output: Credit Status: Good
```

Data Science Relevance: Decision Tree models (like Random Forests and XGBoost) work under the hood as a giant hierarchy of `if-elif-else` checks on feature columns!

Video Resource: IITM BS Lecture: Introduction to the IF Statement



Official lecture explaining logical branching, indentation rules, and conditional execution.

Video Resource: IITM BS Tutorial: If, Else, and Elif Conditions



Hands-on tutorial with multiple practical examples of complex boolean expressions.

2.2 Nested Conditionals and Short-Circuit Evaluation

You can nest `if` blocks inside other `if` blocks. When evaluating compound boolean expressions using `and` and `or`, Python uses **Short-Circuit Evaluation**:

- In `A and B`, if `A` is `False`, Python stops immediately and returns `False` without evaluating `B`.
- In `A or B`, if `A` is `True`, Python stops immediately and returns `True` without evaluating `B`.

2.3 Importing Standard Libraries

Python comes with a rich standard library of useful modules (like `math` and `random`). You can import them using different import syntaxes.

Concept: Library Import Syntaxes

```
import math
print(math.sqrt(16)) # 4.0

import random as rd # Using an alias
print(rd.randint(1, 6)) # Random integer from 1 to 6

from math import pi, log
print(log(10))
```

Video Resource: IITM BS Lecture: Importing Libraries in Python



Official lecture comparing `import module`, `import module as alias`, and `from module import function`.

2.4 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **Leap Year Logic:** A year is a leap year if it is divisible by 4, EXCEPT if it is divisible by 100 (unless it is also divisible by 400). Write a Python script with an `if-else` statement to determine whether `year = 2024` is a leap year.
2. **Short-Circuit Evaluation:** What will be printed by the following code snippet?

```
x = 0
if x != 0 and (10 / x > 2):
    print("Valid")
else:
    print("Safe")
```

Explain why Python does not crash with a `ZeroDivisionError`.

3. **Ternary Operator (Conditional Expression):** In Python, you can write concise 1-line conditionals: `val = A if condition else B`. Convert the following block into a 1-line ternary expression:

```
if score >= 50:
    result = "Pass"
else:
    result = "Fail"
```

4. **Random Sampling:** Write a 3-line script that imports the `random` library and simulates rolling two 6-sided dice, printing their individual values and total sum.
5. **Data Science Application (Anomaly Detection):** An IoT sensor sends temperature readings. If the temperature is between 18°C and 28°C (inclusive), it is "Normal". If it is below 18°C, it is "Warning: Low Temp". If it exceeds 28°C, it is "Alert: Overheating". Write the `if-elif-else` block for `temp = 31.5`.

Chapter 3

Loops & Iteration

In computer science, **Iteration** is the repetition of a process in order to generate a sequence of outcomes. Python provides two primary loop constructs: the `while` loop (condition-driven) and the `for` loop (sequence-driven).

3.1 The while Loop

A `while` loop repeatedly executes a block of code as long as a specified boolean condition remains `True`.

Concept: Syntax of while Loop

```
count = 1
while count <= 5:
    print(f"Iteration: {count}")
    count += 1  # Increment to avoid an infinite loop!
```

Video Resource: IITM BS Lecture: Introduction to While Loop 

Official lecture defining `while` loops, termination criteria, and preventing infinite loops.

3.2 The for Loop and range () Function

Python's `for` loop is designed to iterate directly over the elements of any sequence (such as a string, range, list, or tuple).

Key Pattern / Logic: The range (start, stop, step) Function

- `range(n)` → generates integers from 0 up to $n - 1$.
- `range(a, b)` → generates integers from a up to $b - 1$.
- `range(a, b, s)` → generates integers from a to $b - 1$ stepping by s .

Code Example: Summing Numbers with a for Loop

```
n = 100
total_sum = 0

for i in range(1, n + 1):
    total_sum += i

print(f"Sum of 1 to 100: {total_sum}") # Output: 5050
```

Data Science Relevance: Loops allow data scientists to process rows of datasets, compute iterative optimization steps (like Gradient Descent in Machine Learning), and run Monte Carlo simulations.

Video Resource: IITM BS Lecture: Introduction to For Loop



Official lecture explaining sequence iteration using the `for` loop.

Video Resource: IITM BS Lecture: Range Function and Iteration



Official lecture demonstrating `range()` start, stop, step parameters and iterating over strings directly.

3.3 Nested Loops

A loop can be placed inside another loop. The inner loop completes all its iterations for every single iteration of the outer loop.

Code Example: Generating a Multiplication Table

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i}x{j}={i*j}", end="\t")
    print() # New line after each row
```

Core Intuition: Loop Execution Count and Big- O Time Complexity

- **Single Loop:** A loop running N times executes $O(N)$ operations (Linear Time).
- **Nested Loops:** If an outer loop runs N times and an inner loop runs N times, the total number of body executions is $N \times N = N^2$, yielding $O(N^2)$ time complexity (Quadratic Time).

When working with millions of rows in Data Science, nested loops ($O(N^2)$) become very slow which is why we use vectorized operations in NumPy!

Video Resource: IITM BS Lecture: Nested For Loops



Official lecture demonstrating multi-dimensional iteration using nested loops.

3.4 Loop Control Statements: `break`, `continue`, and `pass`

Syntax / Rule: Loop Controls

- **`break`:** Immediately terminates the loop and jumps to the code outside.
- **`continue`:** Skips the rest of the current iteration and moves directly to the next iteration.
- **`pass`:** A null statement used as a placeholder when syntax requires a statement but no action is needed.

Video Resource: IITM BS Lecture: Break, Continue, and Pass



Official lecture explaining loop control flow modification using `break`, `continue`, and `pass`.

3.5 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **Factorial Calculation:** Write a `while` loop that calculates the factorial of a number $N = 6$ ($6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$).
2. **Pattern Printing:** Write nested `for` loops to print the following triangle pattern for $N = 4$ rows:

```
*  
**  
***  
****
```

3. **Sum of Evens:** Write a `for` loop using `range()` that computes the sum of all *even* numbers between 1 and 50 (inclusive).
4. **Early Stopping (break):** Given a list of search scores `scores = [12, 45, 67, 89, 92, 40]`, write a `for` loop that iterates through scores and prints "Threshold Met: X" for the first score ≥ 80 , then immediately stops using `break`.
5. **Data Science Simulation (Convergence Check):** In optimization, an algorithm updates a parameter x until the difference between consecutive values is smaller than a tolerance $\epsilon = 0.001$. Simulate this using a `while` loop starting with $x = 10.0$, repeatedly updating $x = x * 0.5$ until $x < 0.01$, counting the total iterations taken.

Chapter 4

Lists & Basic Matrix Algorithms

A **List** is an ordered, mutable sequence of items. Lists are the most versatile built-in data structure in Python, capable of holding heterogeneous elements, including other lists (creating 2D and multi-dimensional matrices).

4.1 List Creation, Indexing, and Mutability

Unlike strings (which are immutable), lists are **mutable**, meaning you can change, add, or remove elements in place without creating a new object.

Concept: List Basics

```
numbers = [10, 20, 30, 40, 50]
numbers[0] = 99          # Modify in place: [99, 20, 30, 40, 50]
numbers.append(60)       # Add item to end: [99, 20, 30, 40, 50, 60]
numbers.pop()           # Remove & return last item
print(len(numbers))     # Length of list
```

Core Intuition: List Memory References and Shallow Copy Aliasing

When you assign a list to a new variable (`b = a`), Python creates an **alias** referencing the exact same object in memory! Modifying `b` will unexpectedly modify `a`:

```
a = [1, 2, 3]
b = a          # Reference copy (aliasing)
b[0] = 999     # Modifies both b AND a!

# To create a independent copy, use slicing or copy():
c = a.copy()   # Independent shallow copy
```

Video Resource: IITM BS Lecture: Warmup with Lists 

Official lecture introducing list initialization, indexing, append, and slice assignment.

4.2 Linear Search and Basic Vector Operations

Linear search checks every element sequentially to find a target value or index. In linear algebra, a 1D list represents a **Vector**.

Code Example: Dot Product of Two Vectors

The dot product of two vectors $\mathbf{u} = [u_1, u_2, \dots, u_n]$ and $\mathbf{v} = [v_1, v_2, \dots, v_n]$ is:

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i$$

```
u = [1, 2, 3]
v = [4, 5, 6]

dot_product = 0
for i in range(len(u)):
    dot_product += u[i] * v[i]

print("Dot Product:", dot_product)  # 1*4 + 2*5 + 3*6 = 32
```

Data Science Relevance: Vector dot products form the core of matrix multiplication, cosine similarity, neural network weights calculation ($\mathbf{w}^T \mathbf{x} + b$), and linear regression predictions.

Video Resource: IITM BS Lecture: Dot Product of Vectors 

Official lecture computing vector dot products using `for` loops.

Video Resource: IITM BS Lecture: Naive Linear Search 

Official lecture demonstrating linear search implementation and finding minimum/maximum values.

4.3 2D Lists and Matrix Algorithms

A 2D matrix is represented in Python as a list of lists, where each inner list represents a row.

Key Pattern / Logic: Matrix Operations in Pure Python

Given $A (m \times n)$ and $B (m \times n)$, Matrix Addition $C = A + B$ is defined as $C_{ij} = A_{ij} + B_{ij}$.

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

# Matrix Addition
C = []
for i in range(len(A)):
    row = []
    for j in range(len(A[0])):
        row.append(A[i][j] + B[i][j])
    C.append(row)
# C becomes [[6, 8], [10, 12]]
```

Code Example: Matrix Multiplication ($C = A \times B$)

For $A (m \times k)$ and $B (k \times n)$, $C_{ij} = \sum_{p=1}^k A_{ip}B_{pj}$.

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

C = [[0, 0], [0, 0]]
for i in range(len(A)):
    for j in range(len(B[0])):
        for k in range(len(B)):
            C[i][j] += A[i][k] * B[k][j]
# C becomes [[19, 22], [43, 50]]
```

Video Resource: IITM BS Lecture: Matrix Addition from Scratch



Official lecture implementing 2D matrix addition using nested lists and loops.

Video Resource: IITM BS Lecture: Matrix Multiplication Step-by-Step



Official lecture deriving the 3-loop matrix multiplication algorithm in Python.

4.4 Selection Sort ("Obvious Sort")

Syntax / Rule: Selection Sort Logic

Repeatedly find the minimum element from the unsorted portion of the list and swap it to the beginning.

Video Resource: IITM BS Lecture: The Obvious Sorting Algorithm



Official lecture stepping through selection sort logic and list element swapping.

4.5 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **List Operations:** Start with `data = [15, 8, 42, 4, 23]`:
 - (a) Append 16 to the list.
 - (b) Sort the list in ascending order using `.sort()`.
 - (c) Reverse the sorted list.
2. **Vector Euclidean Distance:** Given two 2D points $\mathbf{p} = [x_1, y_1]$ and $\mathbf{q} = [x_2, y_2]$, write a Python function or script to calculate the Euclidean distance:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

For $\mathbf{p} = [1, 2]$ and $\mathbf{q} = [4, 6]$.

3. **Matrix Transpose:** Write a Python script using nested `for` loops to compute the transpose A^T of a 2×3 matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \implies A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

4. **Manual Selection Sort:** Implement the selection sort algorithm in Python to sort `arr = [64, 25, 12, 22, 11]` without using built-in `.sort()` or `sorted()`.
5. **Data Science Application (Feature Scaling / Min-Max Normalization):** In Machine Learning, feature values x are scaled between 0 and 1 using:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Given `features = [10, 20, 30, 40, 50]`, write a `for` loop to compute the min-max normalized list.

Chapter 5

Tuples, Dictionaries & Sets

Python provides specialized container structures beyond lists, each tailored for performance, immutability, or fast lookup operations: **Tuples** (immutable sequences), **Dictionaries** (key-value hash maps), and **Sets** (unordered unique collections).

5.1 Tuples: Immutable Sequences

A **Tuple** is an ordered, immutable sequence of elements defined using parentheses `()`.

Concept: Tuple Properties

```
point = (10, 20, 30)
print(point[0])    # 10

# Tuple Unpacking
x, y, z = point    # x=10, y=20, z=30

# Immutability: point[0] = 99 raises a TypeError!
```

Data Science Relevance: Tuples guarantee data integrity for fixed coordinates, dataset shapes (`df.shape` returns `(rows, cols)`), and function return values.

Video Resource: IITM BS Lecture: Tuples Explained 

Official lecture explaining tuple immutability, packing, unpacking, and hashability.

5.2 Dictionaries: Key-Value Mapping

A **Dictionary** stores elements in `{key: value}` pairs. Dictionaries provide $\mathcal{O}(1)$ average-time complexity for retrieval by key.

Syntax / Rule: Dictionary Rules

- **Keys must be unique and immutable (hashable):** Strings, numbers, and tuples can be keys; lists cannot.
- **Values can be anything:** Lists, nested dictionaries, functions, objects.

Code Example: Word Frequency Counting

```

text = "apple banana apple orange banana apple"
words = text.split()

counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1

print(counts) # Output: {'apple': 3, 'banana': 2, 'orange': 1}

```

Data Science Relevance: Dictionaries are used to represent JSON API responses, category-to-index mappings in Natural Language Processing (vocabularies), and model hyperparameter configuration dicts.

Video Resource: IITM BS Lecture: Dictionaries for Efficient Storage



Official lecture detailing dictionary creation, `.keys()`, `.values()`, `.items()`, and `.get()` methods.

5.3 Sets: Unordered Unique Collections

A **Set** is an unordered collection of unique elements defined using `{ }` or `set()`.

Key Pattern / Logic: Set Operations

- **Union ($A \cup B$):** Elements in A , B , or both.
- **Intersection ($A \cap B$):** Elements in both A and B .
- **Difference ($A - B$):** Elements in A but not in B .
- **Symmetric Difference ($A \oplus B$):** Elements in A or B but not both.

Video Resource: IITM BS Lecture: Sets and Set Operations



Official lecture explaining set mathematical operations, duplicate removal, and membership testing speed.

5.4 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

- 1. Tuple Operations:** Given `student = ("Alice", 21, "Data Science", 3.8)`:
 - (a) Unpack this tuple into variables `name`, `age`, `major`, and `gpa`.
 - (b) What happens if you try to modify `student[1] = 22`?
- 2. Dictionary Lookup & Updating:** Create a dictionary `stock_prices = {"AAPL": 175.5, "GOOGL": 140.2, "MSFT": 380.0}`:
 - (a) Update the price of "AAPL" to 180.0.
 - (b) Add a new key "AMZN" with price 150.0.
 - (c) Safely fetch the price of "NVDA" using `.get()` without raising a `KeyError`, returning 0.0 if missing.
- 3. Set Membership & Deduplication:** Given a raw customer ID list with duplicates `ids = [101, 102, 101, 105, 102, 108]`:
 - (a) Convert the list to a set to remove all duplicates.
 - (b) Check if customer 105 is in the set.
- 4. Set Algebra:** Given two user cohorts:
$$A = \{\text{"Alice"}, \text{"Bob"}, \text{"Charlie"}\}, \quad B = \{\text{"Bob"}, \text{"David"}, \text{"Alice"}\}$$
 - (a) Find the intersection of A and B .
 - (b) Find the unique users across both cohorts (union).
- 5. Data Science Application (Categorical Encoding Map):** In preprocessing categorical labels into numeric indices for Machine Learning: Given `labels = ["cat", "dog", "cat", "bird", "dog"]`, write a Python snippet to extract unique labels into a dictionary `label2idx` mapping each unique label to a unique integer index (e.g. `{"cat": 0, "dog": 1, "bird": 2}`).

Chapter 6

Functions, Arguments & Scope

A **Function** is a reusable block of code designed to perform a specific task. Functions break down large complex programs into modular, readable, and maintainable units (the DRY principle: Don't Repeat Yourself).

6.1 Defining and Calling Functions

Functions are defined using the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`.

Concept: Function Definition Syntax

```
def greet_user(name):  
    """Docstring: Greet a user by name."""  
    message = f"Hello, {name}! Welcome to Python."  
    return message  
  
# Function Call  
result = greet_user("Alice")  
print(result)
```

Video Resource: IITM BS Lecture: Introduction to Functions



Official lecture defining `def`, parameters, `return` values, and docstrings.

6.2 Types of Function Arguments

Python functions support several types of parameters:

1. **Positional Arguments:** Passed in the exact order declared.
2. **Keyword Arguments:** Passed by explicitly naming parameter names (`name="Alice"`).
3. **Default Arguments:** Parameters with fallback default values if omitted by the caller.
4. **Variable-length Arguments (`*args` and `**kwargs`):**
 - `*args`: Collects extra positional arguments into a tuple.
 - `**kwargs`: Collects extra keyword arguments into a dictionary.

Syntax / Rule: Argument Types Syntax

```
def calculate_price(price, discount=0.1, tax=0.05):
    return price * (1 - discount) * (1 + tax)

print(calculate_price(100))           # Uses defaults: 100 * 0.9 * 1.05 = 94.5
print(calculate_price(100, tax=0.10)) # Keyword arg overrides tax only

def custom_sum(*args):
    return sum(args)                  # args is (1, 2, 3, 4)

print(custom_sum(1, 2, 3, 4))         # 10
```

Data Science Relevance: Machine Learning functions in scikit-learn heavily use default and keyword arguments (e.g., `RandomForestClassifier(n_estimators=100, max_depth=5)`).

Video Resource: IITM BS Lecture: Types of Function Arguments 

Official lecture explaining positional, keyword, default, `*args`, and `**kwargs`.

6.3 Scope of Variables (LEGB Rule)

Scope determines where a variable is accessible in your code. Python resolves variable names using the **LEGB Rule**:

Key Pattern / Logic: The LEGB Rule

1. **L (Local):** Variables defined inside the current function body.
2. **E (Enclosing):** Variables in enclosing/outer functions (nested functions).
3. **G (Global):** Variables defined at the top-level of the script/module.
4. **B (Built-in):** Built-in names in Python (`len`, `sum`, `range`, etc.).

Code Example: Global vs Local Scope

```
counter = 0 # Global Variable

def increment():
    global counter # Explicitly modify global variable
    counter += 1

increment()
print(counter) # Output: 1
```

Video Resource: IITM BS Lecture: Scope of Variables 

Official lecture explaining Local vs Global scope, `global` keyword, and namespace resolution.

6.4 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **Basic Function:** Write a function `is_even(n)` that returns `True` if an integer n is even, and `False` otherwise.
2. **Default Arguments:** Write a function `compute_power(base, exponent=2)` that returns $\text{base}^{\text{exponent}}$. Test it with `compute_power(5)` and `compute_power(2, 3)`.
3. **Flexible Aggregation (*args):** Write a function `calculate_mean(*args)` that accepts any number of numeric arguments and returns their arithmetic mean. Handle the edge case of no arguments passed by returning `0.0`.
4. **Variable Scope:** What will be the output of the following script? Explain why.

```
x = 50

def modify_val():
    x = 100
    print("Inside:", x)

modify_val()
print("Outside:", x)
```

5. **Data Science Application (Min-Max Scaling Function):** Write a modular function `min_max_scaler(data)` that takes a list of numbers, calculates its min and max, and returns a new list of min-max normalized values rounded to 3 decimal places.

Chapter 7

Functional Programming, Comprehensions & Generators

Python supports **Functional Programming** idioms. Instead of explicitly writing multi-line `for` loops, functional Python allows developers to write concise, expressive expressions using list comprehensions, anonymous `lambda` functions, higher-order utilities (`map`, `filter`, `zip`), and memory-efficient `yield` generators.

7.1 List, Dictionary, and Set Comprehensions

Comprehensions provide a compact 1-line syntax to construct new sequences from existing iterables.

Key Pattern / Logic: Comprehension Syntax

List Comprehension: `[expression for item in iterable if condition]`

Dict Comprehension: `{key_expr : val_expr for item in iterable}`

Code Example: Filtering and Transforming

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# List Comprehension: Squares of even numbers
even_squares = [x**2 for x in numbers if x % 2 == 0]
print(even_squares) # Output: [4, 16, 36, 64, 100]

# Dict Comprehension: Number to its cube
cubes = {x: x**3 for x in range(1, 6)}
print(cubes) # Output: {1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
```

Video Resource: IITM BS Lecture: Inline Statements and List Comprehensions



Official lecture explaining syntax, conditional filtering, and nested list comprehensions.

7.2 Lambda Functions, `map()`, `filter()`, `zip()`, and `enumerate()`

Concept: Lambda and Higher-Order Functions

- **Lambda** (`lambda x: expr`): Small anonymous 1-line inline functions.
- **map** (`func, iterable`): Applies `func` to every item in `iterable`.
- **filter** (`predicate, iterable`): Retains items where `predicate(item)` is `True`.
- **zip** (`seq1, seq2`): Pairs elements from multiple sequences element-wise into tuples.
- **enumerate** (`seq`): Yields `(index, item)` pairs during iteration.

Code Example: Functional Pipeline Example

```
nums = [1, 2, 3, 4, 5]

# Square all numbers using map and lambda
squares = list(map(lambda x: x**2, nums)) # [1, 4, 9, 16, 25]

# Filter odd numbers
odds = list(filter(lambda x: x % 2 != 0, nums)) # [1, 3, 5]

# Zip two lists together
names = ["Alice", "Bob"]
scores = [85, 92]
paired = list(zip(names, scores)) # [('Alice', 85), ('Bob', 92)]
```

Data Science Relevance: Feature extraction pipelines (like `df['col'].apply(lambda x: clean(x))`) and model training loops (`for idx, (batch_x, batch_y) in enumerate(dataloader)`) heavily utilize these exact tools.

Video Resource: IITM BS Lecture: Lambda, Enumerate, Zip, Map, and Filter



Official lecture covering functional iteration helpers and lambda syntax.

7.3 Generators and the `yield` Keyword

Unlike regular functions that return a full dataset at once (storing everything in RAM), **Generators** compute items lazily on demand using the `yield` keyword.

Syntax / Rule: Generators vs Functions

A function with `yield` pauses execution after sending a value back and resumes right where it left off when the next item is requested via `next()` or a `for` loop.

Code Example: Infinite Stream Generator

```
def fibonacci_stream():  
    a, b = 0, 1  
    while True:  
        yield a  
        a, b = b, a + b  
  
fib = fibonacci_stream()  
for _ in range(6):  
    print(next(fib), end=" ") # Output: 0 1 1 2 3 5  
print()
```

Video Resource: IITM BS Lecture: Iterators, Generators, and Yield

Official lecture explaining memory optimization, lazy evaluation, and `yield` semantics.

7.4 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **List Comprehension Filtering:** Given `words = ["python", "data", "science", "ai", "machine", "ml"]`, write a 1-line list comprehension to extract words that have a length greater than 3 characters, converted to UPPERCASE.
2. **Lambda Sorting:** Given a list of tuples representing student names and test scores `students = [("Alice", 88), ("Bob", 95), ("Charlie", 78), ("David", 90)]`, sort the list in descending order based on their test scores using `sorted()` and a lambda key.
3. **Zippping Vectors:** Given two lists `x = [1, 2, 3]` and `y = [10, 20, 30]`, write a 1-line list comprehension using `zip(x, y)` to compute the element-wise sum `[11, 22, 33]`.
4. **Fibonacci Generator:** Write a generator function `generate_squares(n)` that yields the squares of numbers from 1 up to n one by one. Iterate over it using a `for` loop.
5. **Data Science Application (Text Preprocessing Pipeline):** In Natural Language Processing (NLP), a raw text string `text = " Data Science and Machine Learning 101 "` needs cleaning. Write a functional pipeline using `map()`, `filter()`, and string methods to:
 - (a) Strip whitespace and convert to lowercase.
 - (b) Tokenize into words.
 - (c) Filter out words that are numbers (using `.isdigit()`).

Chapter 8

File Handling & Exception Management

Data science datasets rarely start in memory—they live on disk as CSV, JSON, text logs, or database dumps. In this chapter, we cover **File I/O** (Input/Output) operations and **Exception Handling** to ensure robust data ingestion pipelines that don't crash when encountering missing or corrupted data.

8.1 Reading and Writing Files

Python uses the built-in `open(filename, mode)` function to interact with files on disk. Using the Context Manager with guarantees that file handles are automatically closed, even if errors occur.

Concept: File Modes and Context Managers

Common modes: 'r' (read), 'w' (write / overwrite), 'a' (append), 'b' (binary mode).

```
# Writing to a text file
with open("data.txt", "w") as f:
    f.write("Line 1: Data Science\n")
    f.write("Line 2: Machine Learning\n")

# Reading line by line (memory efficient)
with open("data.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Video Resource: IITM BS Lecture: Reading and Writing to a File 

Official lecture demonstrating file modes, `open()`, `.read()`, `.readline()`, and `with` context blocks.

8.2 Processing Large Datasets

When log files or text datasets exceed available RAM, loading the entire file with `.read()` causes an Out-Of-Memory (OOM) crash. Instead, stream the file line by line using an iterator or chunked reads.

Syntax / Rule: Line-by-Line Streaming

Iterating over a file object for `line in f`: streams one line at a time into memory ($\mathcal{O}(1)$ memory complexity).

Video Resource: IITM BS Lecture: Handling Very Big Files



Official lecture discussing memory considerations when searching and processing massive files.

8.3 Exception Handling (`try`, `except`, `else`, `finally`)

An **Exception** is a runtime error that halts program execution. Exception handling allows Python to catch runtime errors gracefully.

Key Pattern / Logic: The Exception Handling Block

```
try:
    # Code that might raise an exception
    num = int(input("Enter a number: "))
    result = 100 / num
except ValueError:
    print("Invalid input! Please enter a number.")
except ZeroDivisionError:
    print("Division by zero is undefined.")
else:
    # Executed ONLY if NO exception occurred
    print(f"Result: {result}")
finally:
    # Executed ALWAYS (used for cleanup)
    print("Execution complete.")
```

Data Science Relevance: Web scrapers, data loaders, and model training loops use `try-except` blocks to skip corrupted rows or bad images without crashing overnight training runs.

Video Resource: IITM BS Lecture: Exception Handling Explained



Official lecture explaining `try`, `except`, `finally`, built-in exception types, and raising custom errors.

8.4 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **File Writing & Reading:** Write a Python script that:
 - (a) Creates a file named `log.txt` and writes 3 lines of sample log entries.
 - (b) Reopens `log.txt` in read mode and counts the total number of lines in the file.
2. **Handling Missing Files:** Write a script using `try-except` that attempts to open `"missing_dataset.csv"`. If the file does not exist, catch `FileNotFoundError` and print `"Dataset file not found. Please check filepath."`.
3. **Safe Type Conversion Function:** Write a function `safe_float(value)` that attempts to convert `value` to a float. If a `ValueError` or `TypeError` occurs, return `None`. Test with `"3.14"`, `"abc"`, and `[1, 2]`.
4. **Parsing Log File Data:** Suppose a text file `scores.txt` contains numeric test scores, one per line. Write a script to read the file, convert each line to float inside a `try` block (skipping invalid lines), and calculate the average score.
5. **Data Science Application (CSV Line Counter & Cleaner):** Write a script `clean_csv(input_path, output_path)` that reads a CSV file line by line, removes empty lines or lines starting with `#` (comments), and writes clean lines to `output_path`.

Chapter 9

Recursion & Searching Algorithms

Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem. Combined with Divide-and-Conquer strategy, recursion enables highly efficient algorithms like **Binary Search** ($\mathcal{O}(\log N)$ time complexity).

9.1 The Core Mechanics of Recursion

Every valid recursive function must have two components: 1. **Base Case:** The stopping condition that returns a value directly without further recursive calls (prevents `RecursionError: maximum recursion depth exceeded`). 2. **Recursive Step:** The function calls itself with a reduced or smaller input parameter, moving towards the base case.

Concept: Recursive Factorial

$$N! = \begin{cases} 1 & \text{if } N = 0 \text{ or } N = 1 \quad (\text{Base Case}) \\ N \times (N - 1)! & \text{if } N > 1 \quad (\text{Recursive Step}) \end{cases}$$

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5)) # Output: 120
```

Video Resource: IITM BS Lecture: Theoretical Introduction to Recursion



Official lecture explaining the call stack, activation records, and base cases.

Video Resource: IITM BS Lecture: Recursion Illustration (Sum & Factorial)



Official lecture tracing recursive execution stacks step-by-step.

9.2 Binary Search: Iterative vs Recursive

While Linear Search takes $\mathcal{O}(N)$ time, **Binary Search** searches a **sorted** array in $\mathcal{O}(\log N)$ time by repeatedly dividing the search space in half.

Syntax / Rule: Binary Search Algorithm ($\mathcal{O}(\log N)$)

Compare the target with the middle element `mid`:

- If `target == arr[mid]`, return `mid`.
- If `target < arr[mid]`, search the left sub-array (`high = mid - 1`).
- If `target > arr[mid]`, search the right sub-array (`low = mid + 1`).

Code Example: Iterative and Recursive Binary Search

```
# Iterative Binary Search
def binary_search_iterative(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Recursive Binary Search
def binary_search_recursive(arr, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, high)
    else:
        return binary_search_recursive(arr, target, low, mid - 1)
```

Data Science Relevance: Decision Trees, Decision Trees splitting algorithms (e.g. finding optimal thresholds), and database indexing (B-Trees) use recursive divide-and-conquer principles.

Video Resource: IITM BS Lecture: Introduction to Binary Search



Official lecture explaining the halving technique and $\mathcal{O}(\log N)$ efficiency.

Video Resource: IITM BS Lecture: Recursive Binary Search Implementation



Official lecture implementing binary search recursively in Python.

9.3 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **Recursive Sum:** Write a recursive function `recursive_sum(n)` that calculates the sum of integers from 1 to n .
2. **Recursive String Reversal:** Write a recursive function `reverse_string(s)` that reverses a string without using loops or slicing `[: :-1]`. (Base case: empty string or length 1).
3. **Binary Search Trace:** Given a sorted array `arr = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]`:
 - (a) Trace the values of `low`, `high`, and `mid` when searching for `target = 23`.
 - (b) How many comparison steps were required?
4. **Recursive Fibonacci with Memoization:** The naïve recursive Fibonacci $F(n) = F(n - 1) + F(n - 2)$ takes $\mathcal{O}(2^n)$ exponential time. Write an optimized version `fib_memo(n, memo={})` that uses a dictionary to cache previously computed Fibonacci numbers in $\mathcal{O}(n)$ time.
5. **Data Science Application (Finding Square Root via Binary Search):** Write a function `sqrt_binary_search(tol=1e-5)` that uses binary search on the interval `[0, val]` to find the square root of a non-negative float `val` within tolerance `tol`.

Chapter 10

Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which contain data (attributes) and code (methods). OOP enables modularity, encapsulation, and code reuse when building large software systems.

10.1 Classes, Objects, `__init__`, and `self`

A **Class** is a blueprint for creating objects. An **Object** is an instance of a class.

Concept: Class Definition and Instance Attributes

- `__init__(self, ...)`: The constructor method automatically executed when a new object instance is created. - `self`: Represents the specific instance of the class being created or modified.

```
class Student:
    def __init__(self, name, roll_no, gpa):
        self.name = name          # Instance attribute
        self.roll_no = roll_no
        self.gpa = gpa

    def display_info(self):        # Instance method
        return f"Student: {self.name} | GPA: {self.gpa}"

# Instantiating objects
s1 = Student("Alice", 101, 3.9)
print(s1.display_info())        # Output: Student: Alice | GPA: 3.9
```

Video Resource: IITM BS Lecture: Introduction to OOP Basics



Official lecture introducing class concepts, objects, attributes, and methods.

Video Resource: IITM BS Lecture: Attributes, Methods, Init, and Self



Official lecture explaining the constructor `__init__` and instance binding via `self`.

10.2 Inheritance and Method Overriding

Inheritance allows a child class to inherit attributes and methods from a parent class, promoting code reuse.

Syntax / Rule: Parent vs Child Classes

A child class can override methods from the parent class to specialize behavior, and call `super().__init__()` to reuse parent initialization logic.

Code Example: Model Inheritance Architecture

```
class BaseModel:
    def __init__(self, model_name):
        self.model_name = model_name

    def fit(self, X, y):
        raise NotImplementedError("Subclasses must implement fit()")

class LinearRegressionModel(BaseModel):
    def __init__(self, learning_rate=0.01):
        super().__init__("Linear Regression")
        self.lr = learning_rate

    def fit(self, X, y): # Method Overriding
        print(f"Training {self.model_name} with lr={self.lr}...")

model = LinearRegressionModel(learning_rate=0.05)
model.fit([1, 2], [2, 4])
```

Data Science Relevance: Every major Machine Learning framework in Python including PyTorch (`nn.Module`), scikit-learn (`BaseEstimator`), and TensorFlow (`tf.keras.Model`) is built entirely around OOP class inheritance!

Video Resource: IITM BS Lecture: Inheritance and Method Overriding 

Official lecture demonstrating parent-child class structures, `super()`, and method overriding.

10.3 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **Basic Class Creation:** Design a class `Circle` with an attribute `radius`. Include methods `area()` ($A = \pi r^2$) and `circumference()` ($C = 2\pi r$). Test with `r = 5.0`.
2. **Class Encapsulation:** Create a `BankAccount` class with a private attribute `__balance`. Include methods `deposit(amount)`, `withdraw(amount)` (checking for sufficient funds), and `get_balance()`.
3. **Class vs Instance Variables:** Explain the difference between a class variable and an instance variable in Python. Provide a 5-line code snippet demonstrating a class variable `species = "Canis lupus"` shared across `Dog` instances.
4. **Inheritance Hierarchy:** Create a parent class `Vehicle` with attribute `brand` and method `drive()`. Create a child class `ElectricCar` that inherits from `Vehicle`, adds attribute `battery_capacity`, and overrides `drive()` to print "Driving silently on electricity".
5. **Data Science Application (Custom Transformer Class):** In scikit-learn, data preprocessing transformers implement `.fit()` and `.transform()` methods. Create a class `StandardScaler` with attributes `mean` and `std`. Implement:
 - (a) `fit(data)`: Calculates and stores the mean and standard deviation of a list of floats.
 - (b) `transform(data)`: Returns a new list of z -score normalized values $z = \frac{x - \text{mean}}{\text{std}}$.

Chapter 11

Scientific Computing: NumPy & Pandas

While standard Python lists are versatile, they are slow for massive numerical data because elements are pointers scattered in memory. The scientific Python ecosystem led by **NumPy** and **Pandas** provides contiguous memory layout, vectorization, and fast C-level execution for data manipulation.

11.1 NumPy Arrays vs Python Lists

NumPy (Numerical Python) introduces the `ndarray` (N-dimensional array), an array of homogeneous elements stored contiguously in memory.

Concept: NumPy Arrays and Vectorization

Vectorization allows mathematical operations to operate on entire arrays simultaneously without explicit Python `for` loops.

```
import numpy as np

# Creating NDArrays
arr = np.array([1, 2, 3, 4, 5])

# Vectorized Arithmetic
print(arr * 2)      # Output: [ 2  4  6  8 10]
print(arr ** 2)     # Output: [ 1  4  9 16 25]

# Broadcasting: Matrix + Scalar
matrix = np.ones((2, 3)) # 2x3 matrix of ones
print(matrix + 5)
```

Video Resource: IITM BS Lecture: Introduction to NumPy (Arrays vs Lists)



Official lecture explaining memory layout, performance comparisons, and vectorization benefits.

11.2 Pandas Series and DataFrames

Pandas provides high-performance tabular data structures: the 1D **Series** (labeled column) and the 2D **DataFrame** (tabular dataset with labeled rows and columns).

Syntax / Rule: Pandas DataFrame Mechanics

```
import pandas as pd

# Creating a DataFrame from a Dictionary
data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "Salary": [70000, 80000, 95000]
}
df = pd.DataFrame(data)

# Accessing Columns and Filtering Rows
print(df["Name"])
high_earners = df[df["Salary"] >= 80000]
print(high_earners)
```

Data Science Relevance: Pandas is the default tool for Exploratory Data Analysis (EDA), CSV/Parquet loading (`pd.read_csv()`), handling missing values (`df.fillna()`), and feature engineering.

Video Resource: IITM BS Lecture: Why Pandas? Data Analysis Made Easy 

Official lecture introducing Pandas for data manipulation and tabular alignment.

Video Resource: IITM BS Lecture: Pandas Series and DataFrames 

Official lecture detailing indexing, row selection (`.loc`, `.iloc`), and column operations.

11.3 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **NumPy Vectorization:** Given `arr = np.arange(1, 11)` (integers 1 to 10):
 - (a) Compute a boolean mask for all odd numbers.
 - (b) Use the boolean mask to extract only the odd elements.
 - (c) Replace all even numbers in `arr` with `-1`.
2. **Matrix Statistics with NumPy:** Create a 3×3 matrix with values 1 through 9 using `np.arange(1, 10).reshape(3, 3)`. Calculate:
 - (a) Sum of all elements in the matrix.
 - (b) Column-wise means (`axis=0`).
 - (c) Row-wise maximums (`axis=1`).

3. **Pandas Filtering & Selection:** Given the DataFrame:

```
import pandas as pd
df = pd.DataFrame({
    'City': ['Mumbai', 'Delhi', 'Bangalore', 'Chennai'],
    'Temp': [32, 40, 24, 34],
    'Humidity': [80, 45, 65, 75]
})
```

Filter and print rows where `Temp > 30 AND Humidity > 70`.

4. **Pandas Missing Data Handling:** Create a DataFrame with missing values `np.nan`: `df = pd.DataFrame({'A': [1.0, 2.0, np.nan, 4.0], 'B': [np.nan, 10.0, 20.0, 30.0]})`. Fill missing values in column 'A' with its mean, and column 'B' with 0.0.
5. **Data Science Application (Groupby Aggregation):** Given sales transaction data:

```
sales_df = pd.DataFrame({
    'Category': ['Tech', 'Tech', 'Office', 'Office', 'Tech'],
    'Revenue': [1200, 800, 150, 300, 2000]
})
```

Write a 1-line Pandas expression using `.groupby()` to calculate the total revenue per category.

Chapter 12

Data Visualization & Course Summary

Data Visualization turns raw numerical data into intuitive charts, graphs, and interactive dashboards. In this final chapter, we cover **Matplotlib** (the fundamental plotting library in Python) and review the core programming concepts mastered throughout the course.

12.1 Introduction to Matplotlib

`matplotlib.pyplot` provides a MATLAB-like plotting framework for building line plots, scatter plots, bar charts, histograms, and heatmaps.

Concept: Matplotlib Figure and Axes

A **Figure** represents the entire canvas window. An **Axes** represents an individual plot or subplot inside the figure.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.figure(figsize=(8, 4))
plt.plot(x, y, label="Sine Wave", color="blue", linestyle="--")
plt.title("Trigonometric Sine Function")
plt.xlabel("X values")
plt.ylabel("sin(x)")
plt.legend()
plt.grid(True)
plt.show()
```

Video Resource: IITM BS Lecture: Introduction to Matplotlib



Official lecture demonstrating plot creation, line charts, scatter plots, titles, labels, and legends.

12.2 Common Chart Types for Data Science

Syntax / Rule: Plot Types and Use Cases

- **Line Plot (`plt.plot`):** Trends over continuous time or ordered variables.
- **Scatter Plot (`plt.scatter`):** Relationship and correlation between two numerical variables.
- **Bar Chart (`plt.bar`):** Comparing numerical measurements across discrete categories.
- **Histogram (`plt.hist`):** Frequency distribution of a single continuous variable.

Code Example: Scatter Plot and Histogram Example

```
# Scatter Plot for Feature Association
heights = [160, 165, 170, 175, 180]
weights = [55, 62, 68, 74, 82]
plt.scatter(heights, weights, color='purple')
plt.title("Height vs Weight Scatter Plot")

# Histogram for Distribution Analysis
data = np.random.normal(loc=0, scale=1, size=1000)
plt.hist(data, bins=30, color='teal', edgecolor='black')
plt.title("Normal Distribution Histogram")
```

Data Science Relevance: Data visualization is critical during Exploratory Data Analysis (EDA) to detect outliers, assess skewness, inspect feature correlations, and present findings to business stakeholders.

12.3 Course Summary and Next Steps

Congratulations! You have completed the foundational Python programming curriculum for Data Science.

Key Pattern / Logic: Core Curriculum Overview

1. **Weeks 13:** Variables, Types, Control Flow, and Loops.
2. **Weeks 45:** Data Structures (Lists, Tuples, Dicts, Sets) and Matrix Operations.
3. **Weeks 67:** Modular Design with Functions, Scopes, Comprehensions, and Generators.
4. **Weeks 89:** File Processing, Robust Exception Handling, Recursion, and Binary Search.
5. **Weeks 1012:** Object-Oriented Architecture, Scientific Computing (NumPy/Pandas), and Visualization (Matplotlib).

Video Resource: IITM BS Lecture: Course Summary and Next Steps



Official course summary reviewing core concepts and outlining advanced topics in Data Science.

12.4 Practice Exercises

Detailed step-by-step solutions are available in the Appendix.

1. **Basic Line Plot:** Write a Matplotlib script to plot $y = x^2$ for $x \in [-5, 5]$ with 100 points. Add title "Quadratic Function", x-label "x", y-label " $f(x) = x^2$ ", and display a grid.
2. **Comparing Distributions (Subplots):** Write a Matplotlib script using `plt.subplots(1, 2)` to display two side-by-side plots:
 - (a) Left Subplot: A bar chart of top 3 programming languages ['Python', 'SQL', 'R'] with popularity scores [95, 80, 60].
 - (b) Right Subplot: A histogram of 500 random points drawn from a uniform distribution between 0 and 100.
3. **Customizing Scatter Plots:** Create a scatter plot of 20 random (x, y) points. Set point color to green, marker style to '^' (triangle), and marker size to 50.
4. **Comprehensive Review Question (Data Pipeline):** Write a Python function `analyze_dataset(data)` that takes a list of raw numeric measurements:
 - (a) Cleans the list by removing negative values using list comprehension.
 - (b) Calculates the mean and standard deviation of the valid numbers using NumPy.
 - (c) Returns a dictionary {"count": N, "mean": M, "std": S}.
5. **Data Science Application (Plotting a Loss Curve):** In deep learning, model loss decreases over training epochs. Given `epochs = range(1, 11)` and `loss = [2.5, 1.8, 1.2, 0.8, 0.5, 0.35, 0.25, 0.18, 0.15, 0.12]`, write a Matplotlib script to plot the training loss curve with a red line and square markers.

Index

EDA, [39](#)

Histogram, [39](#)

Matplotlib, [39](#)

Scatter Plot, [39](#)

Solutions to Practice Exercises

This appendix contains step-by-step solutions and explanations for all practice exercises across Weeks 1 to 12.

Week 1: Introduction to Python, Variables, Data Types & Operators

Detailed Solutions

1. Variable Types

- **(a)** `10 / 2`: **float**. In Python 3, standard division `/` always returns a floating-point number (`5.0`), even if the result is a whole number.
- **(b)** `10 // 2`: **int**. Floor division `//` truncates the fractional part and returns an integer (`5`).
- **(c)** `"100" + "200"`: **str**. Using `+` on two strings performs concatenation, producing `"100200"`.
- **(d)** `(5 > 3) and (2 == 4)`: **bool**. `5 > 3` evaluates to `True`, and `2 == 4` evaluates to `False`. `True and False` evaluates to `False`.

2. Arithmetic Logic

- **(a)** `2 + 3 * 4 ** 2`: Exponentiation has highest precedence ($4^2 = 16$). Next is multiplication ($3 \times 16 = 48$). Finally addition ($2 + 48 = 50$).
- **(b)** `25 % 7`: Modulus gives the remainder of $25 \div 7$. $7 \times 3 = 21$, leaving a remainder of 4.
- **(c)** `19 // 4`: Floor division of $19 \div 4 = 4.75$, dropping the decimal gives 4.

3. String Slicing

- **(a)** `text[0:7]` or `text[:7]` returns `"Machine"`.
- **(b)** `text[7:]` returns `"Learning"`.
- **(c)** `text[-8:]` extracts the last 8 characters, which is `"Learning"`.
- **(d)** `text[::-1]` reverses the string to `"gninraeLEnihcaM"`.

4. Type Casting & Input

```
age_str = "22"
future_age = int(age_str) + 5
print(f"In 5 years, you will be {future_age}")
```

5. Data Science Application: BMI Calculation

```
weight = 75
height = 1.75
bmi = round(weight / (height ** 2), 2)
print("Patient BMI:", bmi) # Output: Patient BMI: 24.49
```

Week 2: Control Flow & Conditionals

Detailed Solutions

1. Leap Year Logic

```
year = 2024
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(f"{year} is a Leap Year")
else:
    print(f"{year} is NOT a Leap Year")
```

2. Short-Circuit Evaluation The output is **Safe**. Because $x = 0$, the first operand $x \neq 0$ evaluates to `False`. Due to short-circuit evaluation, Python immediately knows the entire and expression is `False` without evaluating the second term ($10 / x > 2$). Thus, division by zero never occurs, preventing a runtime crash.

3. Ternary Operator

```
result = "Pass" if score >= 50 else "Fail"
```

4. Random Sampling

```
import random
d1, d2 = random.randint(1, 6), random.randint(1, 6)
print(f"Die 1: {d1}, Die 2: {d2}, Total: {d1 + d2}")
```

5. Data Science Application: Anomaly Detection

```
temp = 31.5
if 18 <= temp <= 28:
    status = "Normal"
elif temp < 18:
    status = "Warning: Low Temp"
else:
    status = "Alert: Overheating"

print(status) # Output: Alert: Overheating
```

Week 3: Loops & Iteration

Detailed Solutions

1. Factorial Calculation

```
N = 6
factorial = 1
curr = N
while curr > 0:
    factorial *= curr
    curr -= 1

print(f"{N}! = {factorial}") # Output: 6! = 720
```

2. Pattern Printing

```
N = 4
for i in range(1, N + 1):
    for j in range(i):
        print("*", end=" ")
    print()
```

3. Sum of Evens

```
even_sum = 0
for i in range(2, 51, 2):
    even_sum += i

print(f"Sum of evens from 1 to 50: {even_sum}") # Output: 650
```

4. Early Stopping (break)

```
scores = [12, 45, 67, 89, 92, 40]
for score in scores:
    if score >= 80:
        print(f"Threshold Met: {score}") # Prints 89
        break
```

5. Data Science Simulation: Convergence Check

```
x = 10.0
iterations = 0
while x >= 0.01:
    x *= 0.5
    iterations += 1

print(f"Converged in {iterations} iterations. Final x: {x}")
# Output: Converged in 10 iterations. Final x: 0.009765625
```

Week 4: Lists & Basic Matrix Algorithms

Detailed Solutions

1. List Operations

```
data = [15, 8, 42, 4, 23]
data.append(16) # [15, 8, 42, 4, 23, 16]
data.sort() # [4, 8, 15, 16, 23, 42]
data.reverse() # [42, 23, 16, 15, 8, 4]
print(data)
```

2. Vector Euclidean Distance

```
import math
p = [1, 2]
q = [4, 6]

dist = math.sqrt((p[0] - q[0])**2 + (p[1] - q[1])**2)
print("Euclidean Distance:", dist) # Output: 5.0
```

3. Matrix Transpose

```
A = [[1, 2, 3],
      [4, 5, 6]]

AT = []
for j in range(len(A[0])): # Columns of A become rows of AT
    row = []
    for i in range(len(A)):
        row.append(A[i][j])
    AT.append(row)

print("A Transposed:", AT) # Output: [[1, 4], [2, 5], [3, 6]]
```

4. Manual Selection Sort


```
arr = [64, 25, 12, 22, 11]
n = len(arr)

for i in range(n):
    min_idx = i
    for j in range(i + 1, n):
        if arr[j] < arr[min_idx]:
            min_idx = j
    # Swap minimum found with first unsorted element
    arr[i], arr[min_idx] = arr[min_idx], arr[i]

print("Sorted Array:", arr) # Output: [11, 12, 22, 25, 64]
```

5. Data Science Application: Min-Max Normalization

```
features = [10, 20, 30, 40, 50]
x_min = min(features)
x_max = max(features)

scaled = []
for x in features:
    x_scaled = (x - x_min) / (x_max - x_min)
    scaled.append(round(x_scaled, 2))

print("Normalized Features:", scaled)
# Output: [0.0, 0.25, 0.5, 0.75, 1.0]
```

Week 5: Tuples, Dictionaries & Sets

Detailed Solutions

1. Tuple Operations

```
student = ("Alice", 21, "Data Science", 3.8)
name, age, major, gpa = student

# Trying to modify student[1] = 22 raises:
# TypeError: 'tuple' object does not support item assignment
```

2. Dictionary Lookup & Updating

```
stock_prices = {"AAPL": 175.5, "GOOGL": 140.2, "MSFT": 380.0}

stock_prices["AAPL"] = 180.0
stock_prices["AMZN"] = 150.0

nvda_price = stock_prices.get("NVDA", 0.0)
print("NVDA Price:", nvda_price) # Output: 0.0
```

3. Set Membership & Deduplication

```
ids = [101, 102, 101, 105, 102, 108]
unique_ids = set(ids)

print("Unique IDs:", unique_ids) # {101, 102, 105, 108}
print("Is 105 present?", 105 in unique_ids) # True
```

4. Set Algebra

```
A = {"Alice", "Bob", "Charlie"}
B = {"Bob", "David", "Alice"}
```

```
intersection = A & B # {"Alice", "Bob"}
union = A | B      # {"Alice", "Bob", "Charlie", "David"}

print("Both Cohorts:", intersection)
print("All Unique Cohort Members:", union)
```

5. Data Science Application: Categorical Encoding Map

```
labels = ["cat", "dog", "cat", "bird", "dog"]

label2idx = {}
for label in labels:
    if label not in label2idx:
        label2idx[label] = len(label2idx)

print("Label Encoding Map:", label2idx)
# Output: {'cat': 0, 'dog': 1, 'bird': 2}
```

Week 6: Functions, Arguments & Scope

Detailed Solutions

1. Basic Function

```
def is_even(n):
    return n % 2 == 0

print(is_even(4)) # True
print(is_even(7)) # False
```

2. Default Arguments

```
def compute_power(base, exponent=2):
    return base ** exponent

print(compute_power(5))      # Output: 25 (5^2)
print(compute_power(2, 3))   # Output: 8 (2^3)
```

3. Flexible Aggregation (*args)

```
def calculate_mean(*args):
    if not args:
        return 0.0
    return sum(args) / len(args)

print(calculate_mean(10, 20, 30, 40)) # Output: 25.0
print(calculate_mean())                # Output: 0.0
```

4. Variable Scope

```
# Output:
# Inside: 100
# Outside: 50
```

Explanation: Inside `modify_val()`, the assignment `x = 100` creates a new *local* variable `x` within the function scope. It shadows the global `x` without modifying it. Once the function finishes, the local variable is destroyed, and the global variable `x` remains 50.

5. Data Science Application: Min-Max Scaler Function

```
def min_max_scaler(data):
    if not data:
        return []
```

```

x_min = min(data)
x_max = max(data)
if x_min == x_max:
    return [0.0] * len(data)

return [round((x - x_min) / (x_max - x_min), 3) for x in data]

sample_data = [10, 25, 50, 75, 100]
print(min_max_scaler(sample_data))
# Output: [0.0, 0.167, 0.444, 0.722, 1.0]

```

Week 7: Functional Programming, Comprehensions & Generators

Detailed Solutions

1. List Comprehension Filtering

```

words = ["python", "data", "science", "ai", "machine", "ml"]
filtered = [w.upper() for w in words if len(w) > 3]

print(filtered) # Output: ['PYTHON', 'DATA', 'SCIENCE', 'MACHINE']

```

2. Lambda Sorting

```

students = [("Alice", 88), ("Bob", 95), ("Charlie", 78), ("David", 90)]

sorted_students = sorted(students, key=lambda student: student[1], reverse=True)
print(sorted_students)
# Output: [('Bob', 95), ('David', 90), ('Alice', 88), ('Charlie', 78)]

```

3. Zipping Vectors

```

x = [1, 2, 3]
y = [10, 20, 30]

elementwise_sum = [a + b for a, b in zip(x, y)]
print(elementwise_sum) # Output: [11, 22, 33]

```

4. Squares Generator

```

def generate_squares(n):
    for i in range(1, n + 1):
        yield i ** 2

for val in generate_squares(5):
    print(val, end=" ") # Output: 1 4 9 16 25
print()

```

5. Data Science Application: Text Preprocessing Pipeline

```

text = "    Data Science and Machine Learning 101    "

# 1 & 2. Strip whitespace, lower case, tokenize
tokens = text.strip().lower().split()

# 3. Filter out numeric tokens
clean_tokens = list(filter(lambda w: not w.isdigit(), tokens))

print(clean_tokens)
# Output: ['data', 'science', 'and', 'machine', 'learning']

```

Week 8: File Handling & Exception Management

Detailed Solutions

1. File Writing & Reading

```
# Part A: Writing log lines
with open("log.txt", "w") as f:
    f.write("2026-07-26 INFO Model training started\n")
    f.write("2026-07-26 INFO Epoch 1 complete\n")
    f.write("2026-07-26 INFO Model saved\n")

# Part B: Counting lines
with open("log.txt", "r") as f:
    line_count = sum(1 for _ in f)

print(f"Total lines in log.txt: {line_count}") # Output: 3
```

2. Handling Missing Files

```
try:
    with open("missing_dataset.csv", "r") as f:
        content = f.read()
except FileNotFoundError:
    print("Dataset file not found. Please check filepath.")
```

3. Safe Type Conversion Function

```
def safe_float(value):
    try:
        return float(value)
    except (ValueError, TypeError):
        return None

print(safe_float("3.14")) # 3.14
print(safe_float("abc")) # None
print(safe_float([1, 2])) # None
```

4. Parsing Log File Data

```
total_score = 0.0
valid_count = 0

with open("scores.txt", "r") as f:
    for line in f:
        try:
            val = float(line.strip())
            total_score += val
            valid_count += 1
        except ValueError:
            # Skip invalid header or corrupted text lines
            continue

if valid_count > 0:
    avg = total_score / valid_count
    print(f"Average Score: {avg:.2f}")
```

5. Data Science Application: CSV Line Cleaner

```
def clean_csv(input_path, output_path):
    cleaned_lines = 0
    with open(input_path, "r") as fin, open(output_path, "w") as fout:
        for line in fin:
            stripped = line.strip()
            # Ignore empty lines and comment lines
```

```

        if stripped and not stripped.startswith("#"):
            fout.write(line)
            cleaned_lines += 1
    print(f"Cleaned CSV written with {cleaned_lines} valid rows.")

```

Week 9: Recursion & Searching Algorithms

Detailed Solutions

1. Recursive Sum

```

def recursive_sum(n):
    if n <= 1:
        return n
    return n + recursive_sum(n - 1)

print(recursive_sum(10)) # Output: 55

```

2. Recursive String Reversal

```

def reverse_string(s):
    if len(s) <= 1:
        return s
    return s[-1] + reverse_string(s[:-1])

print(reverse_string("python")) # Output: "nohtyp"

```

3. Binary Search Trace Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91], Target: 23.

- Step 1:** low = 0, high = 9. mid = (0 + 9) // 2 = 4. arr[4] = 16. Since 23 > 16, search right half: low = 5.
- Step 2:** low = 5, high = 9. mid = (5 + 9) // 2 = 7. arr[7] = 56. Since 23 < 56, search left half: high = 6.
- Step 3:** low = 5, high = 6. mid = (5 + 6) // 2 = 5. arr[5] = 23. Match found!

Total Comparisons: 3 comparison steps required.

4. Recursive Fibonacci with Memoization

```

def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    if n <= 1:
        return n

    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

print(fib_memo(50)) # Output: 12586269025 (computes instantly!)

```

5. Data Science Application: Square Root via Binary Search

```

def sqrt_binary_search(val, tol=1e-5):
    if val < 0:
        return None
    low, high = 0.0, max(1.0, val)

    while high - low > tol:
        mid = (low + high) / 2.0

```

```
        if mid * mid < val:
            low = mid
        else:
            high = mid

    return round((low + high) / 2.0, 5)

print("sqrt(25) =", sqrt_binary_search(25))    # Output: 5.0
print("sqrt(2) =", sqrt_binary_search(2))      # Output: 1.41421
```

Week 10: Object-Oriented Programming (OOP)

Detailed Solutions

1. Basic Class Creation

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return round(math.pi * (self.radius ** 2), 2)

    def circumference(self):
        return round(2 * math.pi * self.radius, 2)

c = Circle(5.0)
print("Area:", c.area())                # Output: 78.54
print("Circumference:", c.circumference()) # Output: 31.42
```

2. Class Encapsulation

```
class BankAccount:
    def __init__(self, balance=0.0):
        self.__balance = balance # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return True
        return False

    def get_balance(self):
        return self.__balance

acc = BankAccount(100.0)
acc.deposit(50.0)
acc.withdraw(30.0)
print("Balance:", acc.get_balance())    # Output: 120.0
```

3. Class vs Instance Variables Class variables are shared by all instances of a class. Instance variables are bound to a specific object instance.

```
class Dog:
    species = "Canis lupus" # Class Variable shared across all dogs
```

```

def __init__(self, name):
    self.name = name      # Instance Variable unique to each dog

d1, d2 = Dog("Buddy"), Dog("Max")
print(d1.species, d1.name) # Canis lupus Buddy
print(d2.species, d2.name) # Canis lupus Max

```

4. Inheritance Hierarchy

```

class Vehicle:
    def __init__(self, brand):
        self.brand = brand

    def drive(self):
        print(f"Driving {self.brand} vehicle...")

class ElectricCar(Vehicle):
    def __init__(self, brand, battery_capacity):
        super().__init__(brand)
        self.battery_capacity = battery_capacity

    def drive(self): # Method Overriding
        print(f"Driving {self.brand} silently on {self.battery_capacity} kWh battery")

tesla = ElectricCar("Tesla", 85)
tesla.drive() # Output: Driving Tesla silently on 85 kWh battery

```

5. Data Science Application: Custom StandardScaler Class

```

import math

class StandardScaler:
    def __init__(self):
        self.mean = 0.0
        self.std = 1.0

    def fit(self, data):
        self.mean = sum(data) / len(data)
        variance = sum((x - self.mean) ** 2 for x in data) / len(data)
        self.std = math.sqrt(variance)

    def transform(self, data):
        if self.std == 0:
            return [0.0] * len(data)
        return [round((x - self.mean) / self.std, 3) for x in data]

scaler = StandardScaler()
train_data = [10.0, 20.0, 30.0, 40.0, 50.0]
scaler.fit(train_data)

scaled = scaler.transform(train_data)
print("Standardized Data:", scaled)
# Output: [-1.414, -0.707, 0.0, 0.707, 1.414]

```

Week 11: Scientific Computing: NumPy & Pandas

Detailed Solutions

1. NumPy Vectorization

```

import numpy as np

```

```

arr = np.arange(1, 11)

# Part A: Boolean mask for odds
mask = (arr % 2 != 0)

# Part B: Extract odds
odds = arr[mask]
print("Odds:", odds) # [1, 3, 5, 7, 9]

# Part C: Replace evens with -1
arr[arr % 2 == 0] = -1
print("Modified Array:", arr) # [ 1 -1  3 -1  5 -1  7 -1  9 -1]

```

2. Matrix Statistics with NumPy

```

mat = np.arange(1, 10).reshape(3, 3)

total_sum = mat.sum()
col_means = mat.mean(axis=0)
row_maxs = mat.max(axis=1)

print("Total Sum:", total_sum) # 45
print("Column Means:", col_means) # [4. 5. 6.]
print("Row Maxima:", row_maxs) # [3 6 9]

```

3. Pandas Filtering & Selection

```

import pandas as pd

df = pd.DataFrame({
    'City': ['Mumbai', 'Delhi', 'Bangalore', 'Chennai'],
    'Temp': [32, 40, 24, 34],
    'Humidity': [80, 45, 65, 75]
})

filtered_df = df[(df['Temp'] > 30) & (df['Humidity'] > 70)]
print(filtered_df)
# Output:
#      City  Temp  Humidity
# 0  Mumbai   32         80
# 3  Chennai   34         75

```

4. Pandas Missing Data Handling

```

import pandas as pd
import numpy as np

df = pd.DataFrame({
    'A': [1.0, 2.0, np.nan, 4.0],
    'B': [np.nan, 10.0, 20.0, 30.0]
})

df['A'] = df['A'].fillna(df['A'].mean())
df['B'] = df['B'].fillna(0.0)

print(df)
# Output:
#      A      B
# 0  1.000000  0.0
# 1  2.000000 10.0
# 2  2.333333 20.0
# 3  4.000000 30.0

```

5. Data Science Application: Groupby Aggregation


```
sales_df = pd.DataFrame({
    'Category': ['Tech', 'Tech', 'Office', 'Office', 'Tech'],
    'Revenue': [1200, 800, 150, 300, 2000]
})

total_rev = sales_df.groupby('Category')['Revenue'].sum()
print(total_rev)
# Output:
# Category
# Office      450
# Tech       4000
# Name: Revenue, dtype: int64
```

Week 12: Data Visualization & Course Summary

Detailed Solutions

1. Basic Line Plot

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-5, 5, 100)
y = x ** 2

plt.figure(figsize=(6, 4))
plt.plot(x, y, color='blue', linewidth=2)
plt.title("Quadratic Function")
plt.xlabel("x")
plt.ylabel("f(x) = x^2")
plt.grid(True)
plt.show()
```

2. Comparing Distributions (Subplots)

```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Subplot 1: Bar Chart
langs = ['Python', 'SQL', 'R']
scores = [95, 80, 60]
axes[0].bar(langs, scores, color='skyblue', edgecolor='black')
axes[0].set_title("Language Popularity")

# Subplot 2: Histogram
data = np.random.uniform(0, 100, 500)
axes[1].hist(data, bins=20, color='orange', edgecolor='black')
axes[1].set_title("Uniform Distribution")

plt.tight_layout()
plt.show()
```

3. Customizing Scatter Plots

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
x = np.random.rand(20)
```

```
y = np.random.rand(20)

plt.scatter(x, y, color='green', marker='^', s=50)
plt.title("Green Triangle Scatter Plot")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()
```

4. Comprehensive Review Question: Data Pipeline Function

```
import numpy as np

def analyze_dataset(data):
    # 1. Clean negative numbers using list comprehension
    clean_data = [x for x in data if x >= 0]

    if not clean_data:
        return {"count": 0, "mean": 0.0, "std": 0.0}

    # 2. Convert to NumPy for stats
    arr = np.array(clean_data)

    # 3. Return summary dict
    return {
        "count": len(arr),
        "mean": round(float(arr.mean()), 2),
        "std": round(float(arr.std()), 2)
    }

raw_measurements = [12.5, -4.0, 18.2, 22.0, -1.0, 15.8]
print(analyze_dataset(raw_measurements))
# Output: {'count': 4, 'mean': 17.12, 'std': 3.42}
```

5. Data Science Application: Plotting a Loss Curve

```
import matplotlib.pyplot as plt

epochs = list(range(1, 11))
loss = [2.5, 1.8, 1.2, 0.8, 0.5, 0.35, 0.25, 0.18, 0.15, 0.12]

plt.figure(figsize=(7, 4))
plt.plot(epochs, loss, 'r-s', label="Training Loss", linewidth=2)
plt.title("Deep Learning Model Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Cross-Entropy Loss")
plt.legend()
plt.grid(True)
plt.show()
```